PYTHON LAB PROGRAMS

Complete Codes and Outputs

Includes Linear Regression, A* Search, Support Vector Machine (SVM) Classification, and Breadth-First Search (BFS) / Depth-First Search (DFS).

Note: Some numerical outputs depend on the dataset and installed library versions. Where applicable, the PDF identifies those outputs as variable.

1. Salary Prediction Using Linear Regression

Fits a linear regression model to predict salary from years of experience, calculates evaluation metrics, and plots the regression line.

Python Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

# Create salary_data.csv with the columns shown below.
df = pd.read_csv("salary_data.csv")

print("=== SALARY DATASET ===")
print(df.head())
print("Rows:", len(df))

X = df[["Experience Years"]]
y = df["Salary"]

# Test Case 1: Train model and check positive slope
model = LinearRegression()
model.fit(X, y)

print("\n=== TEST CASE 1 ===")
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("RESULT:", "PASS" if model.coef_[0] > 0 else "FAIL")

# Test Case 2: Test-set prediction and metrics
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
m = LinearRegression()
m.fit(X_train, y_train)
pred = m.predict(X_test)

mse = mean_squared_error(y_test, pred)
r2 = r2_score(y_test, pred)

print("\n=== TEST CASE 2 ===")
print("Actual:", y_test.values)
print("Predicted:", np.round(pred, 2))
print("MSE:", round(mse, 2))
print("R2:", round(r2, 3))
print("RESULT:", "PASS" if r2 > 0.5 else "FAIL")

# Test Case 3: Predict salary for 7 years of experience
salary_7 = model.predict(pd.DataFrame({"Experience Years": [7]}))[0]
print("\n=== TEST CASE 3 ===")
print("7 Years Salary:", round(salary_7, 2))

# Test Case 4: Check constant-target behavior
constant_model = LinearRegression()
constant_model.fit(X, np.ones(len(y)))
print("\n=== TEST CASE 4 ===")
print("Constant Target Slope:", constant_model.coef_[0])
print("RESULT:", "PASS" if abs(constant_model.coef_[0]) < 1e-6 else "FAIL")

print("\n=== TEST CASE 5 ===")
print("Normal MSE:", round(mse, 2))

plt.scatter(X, y, label="Data")
plt.plot(X, model.predict(X), label="Regression Line")
plt.xlabel("Experience Years")
plt.ylabel("Salary")
plt.title("Salary vs Experience")
plt.legend()
plt.show()
```

Expected Output

```
=== SALARY DATASET ===
(Displays first five rows of salary_data.csv)
Rows: depends on the CSV file

=== TEST CASE 1 ===
```

```
Slope: depends on the dataset
Intercept: depends on the dataset
RESULT: PASS/FAIL depends on the slope

=== TEST CASE 2 ===
Actual: (test-set salary values)
Predicted: (model predictions)
MSE: (calculated value)
R2: (calculated value)
RESULT: PASS/FAIL depends on R2

=== TEST CASE 3 ===
7 Years Salary: (predicted value)

=== TEST CASE 4 ===
Constant Target Slope: 0.0
RESULT: PASS

=== TEST CASE 5 ===
Normal MSE: (same MSE calculated in Test Case 2)

A regression graph is displayed.
```

CSV file required: Save a file named **salary_data.csv** in the same folder. It must contain the columns **Experience Years** and **Salary**. Example rows: 1.1,39343; 1.2,42774; 1.3,46205; 1.5,37731; 2.0,43525. Use the complete dataset from your lab record for matching results. Install dependencies if needed: pandas, numpy, matplotlib, scikit-learn.

2. A* Search Algorithm

Finds a path from a start node to a goal node using edge costs and heuristic estimates.

Python Code

```
def a_star(graph, h, start, goal):
    open_list = [start]
    closed = set()
    g = {start: 0}
    parent = {start: start}

    while open_list:
        n = min(open_list, key=lambda node: g[node] + h[node])

        if n == goal:
            path = []
            current = n
            while parent[current] != current:
                path.append(current)
                current = parent[current]
            path.append(start)
            path.reverse()
            print("Path:", path)
            print("Cost:", g[goal])
            return path, g[goal]

        open_list.remove(n)
        closed.add(n)

        for m, cost in graph[n].items():
            new_cost = g[n] + cost
            if m not in g or new_cost < g[m]:
                g[m] = new_cost
                parent[m] = n
                if m in closed:
                    closed.remove(m)
                if m not in open_list:
                    open_list.append(m)

    print("Path not found")
    return None, float("inf")


print("Test Case 1")
graph = {
    'A': {'B': 1, 'C': 3, 'G': 2},
    'B': {'E': 2},
    'C': {'G': 3},
    'E': {'G': 3},
    'G': {}
}
h = {'A': 2, 'B': 3, 'C': 1, 'E': 1, 'G': 0}
a_star(graph, h, 'A', 'G')

print("\nTest Case 2")
graph = {
    'S': {'A': 2, 'B': 3},
    'A': {'G': 4},
    'B': {'G': 3},
    'G': {}
}
h = {'S': 5, 'A': 4, 'B': 2, 'G': 0}
a_star(graph, h, 'S', 'G')

print("\nTest Case 3")
graph = {'A': {'G': 2}, 'G': {}}
h = {'A': 1, 'G': 0}
a_star(graph, h, 'A', 'G')

print("\nTest Case 4")
graph = {'A': {'B': 2}, 'B': {}, 'G': {}}
h = {'A': 2, 'B': 1, 'G': 0}
a_star(graph, h, 'A', 'G')

print("\nTest Case 5")
graph = {
    'S': {'A': 1, 'B': 2},
    'A': {'G': 10},
    'B': {'G': 3},
    'G': {}
}
h = {'S': 0, 'A': 0, 'B': 100, 'G': 0}
```

```
a_star(graph, h, 'S', 'G')
```

Expected Output

```
Test Case 1
Path: ['A', 'G']
Cost: 2

Test Case 2
Path: ['S', 'B', 'G']
Cost: 6

Test Case 3
Path: ['A', 'G']
Cost: 2

Test Case 4
Path not found

Test Case 5
Path: ['S', 'A', 'G']
Cost: 11
```

A* chooses the node with the smallest f(n) = g(n) + h(n). In Test Case 5, the deliberately misleading heuristic makes the algorithm stop at a non-optimal goal path; the cheaper path through B has cost 5, but this heuristic is not admissible.

3. Support Vector Machine (SVM) Classification

Classifies Iris data, reports accuracy and a decision value, tests a single-class error case, and plots a linear decision boundary.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target

# TC1: Two linearly separable classes
X1, y1 = X[y != 2], y[y != 2]
model1 = SVC(kernel="linear")
model1.fit(X1, y1)
print("TC1 Accuracy:", accuracy_score(y1, model1.predict(X1)))

# TC2: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)
model2 = SVC(kernel="linear")
model2.fit(X_train, y_train)
print("TC2 Accuracy:", accuracy_score(y_test, model2.predict(X_test)))

# TC3: Predict one sample and show its decision value
model3 = SVC(kernel="linear")
model3.fit(X1, y1)
point = X1[0].reshape(1, -1)
print("TC3 Predicted class:", model3.predict(point)[0])
print("TC3 Decision value:", model3.decision_function(point)[0])

# TC4: Add noise to make the classes overlap
rng = np.random.default_rng(42)
Xu = X1 + rng.normal(0, 0.8, X1.shape)
model4 = SVC(kernel="linear")
model4.fit(Xu, y1)
print("TC4 Accuracy:", accuracy_score(y1, model4.predict(Xu)))

# TC5: SVM requires at least two classes
X5, y5 = X[y == 0], y[y == 0]
try:
    model5 = SVC(kernel="linear")
    model5.fit(X5, y5)
except ValueError:
    print("TC5: SVM needs at least two classes.")

# Graph using sepal length and sepal width
Xg = X1[:, [0, 1]]
graph_model = SVC(kernel="linear")
graph_model.fit(Xg, y1)

plt.scatter(Xg[y1 == 0, 0], Xg[y1 == 0, 1], label="Class 0")
plt.scatter(Xg[y1 == 1, 0], Xg[y1 == 1, 1], label="Class 1")

xx = np.linspace(Xg[:, 0].min(), Xg[:, 0].max(), 100)
w = graph_model.coef_[0]
b = graph_model.intercept_[0]
yy = -(w[0] * xx + b) / w[1]
plt.plot(xx, yy, label="Hyperplane")

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("SVM Classification")
plt.legend()
plt.grid()
plt.show()
```

Expected Output

```
TC1 Accuracy: 1.0
TC2 Accuracy: 1.0
TC3 Predicted class: 0
TC3 Decision value: depends on the selected sample
TC4 Accuracy: depends on the generated noisy data
```

```
TC5: SVM needs at least two classes.

A graph is displayed showing the two classes and the linear hyperplane.
```

The Iris dataset is included in scikit-learn, so no CSV file is required. TC4 uses a fixed random seed to make the added noise reproducible. Install dependencies if needed: scikit-learn, numpy, matplotlib.

4. BFS and DFS Graph Traversal

Traverses connected, branching, cyclic, single-node, and disconnected graphs using BFS and DFS.

Python Code

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    traversal = []

    while queue:
        node = queue.popleft()
        traversal.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return traversal


def dfs(graph, start):
    visited = set()
    traversal = []

    def dfs_util(node):
        visited.add(node)
        traversal.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs_util(neighbor)

    dfs_util(start)
    return traversal


# Test Case 1: Connected graph
graph1 = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C']
}
print("Test Case 1: Connected graph")
print("BFS:", bfs(graph1, 'A'))
print("DFS:", dfs(graph1, 'A'))

# Test Case 2: Multiple branches
graph2 = {
    'A': ['B', 'C'],
    'B': ['E', 'F'],
    'C': ['D', 'G'],
    'D': ['H'],
    'E': [], 'F': [], 'G': [], 'H': [], 'I': []
}
print("\\nTest Case 2: Multiple branches")
print("BFS:", bfs(graph2, 'A'))
print("DFS:", dfs(graph2, 'A'))

# Test Case 3: Cyclic graph
graph3 = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
print("\\nTest Case 3: Cyclic graph")
print("BFS:", bfs(graph3, 'A'))
print("DFS:", dfs(graph3, 'A'))

# Test Case 4: Single-node graph
graph4 = {'A': []}
print("\\nTest Case 4: Single-node graph")
print("BFS:", bfs(graph4, 'A'))
print("DFS:", dfs(graph4, 'A'))

# Test Case 5: Disconnected graph
graph5 = {
    'A': ['B'], 'B': ['A'],
```

```
    'C': ['D'], 'D': ['C'], 'E': []
}
print("\\nTest Case 5: Disconnected graph")
print("BFS:", bfs(graph5, 'A'))
print("DFS:", dfs(graph5, 'A'))
```

Expected Output

```
Test Case 1: Connected graph
BFS: ['A', 'B', 'C', 'D', 'E', 'F']
DFS: ['A', 'B', 'D', 'E', 'C', 'F']

Test Case 2: Multiple branches
BFS: ['A', 'B', 'C', 'E', 'F', 'D', 'G', 'H']
DFS: ['A', 'B', 'E', 'F', 'C', 'D', 'H', 'G']

Test Case 3: Cyclic graph
BFS: ['A', 'B', 'C', 'D']
DFS: ['A', 'B', 'D', 'C']

Test Case 4: Single-node graph
BFS: ['A']
DFS: ['A']

Test Case 5: Disconnected graph
BFS: ['A', 'B']
DFS: ['A', 'B']
```

BFS visits nodes level by level using a queue. DFS explores a branch deeply before backtracking. For a disconnected graph, these functions return only the nodes reachable from the chosen start node. Traversal order depends on the order of neighbors in the adjacency lists.

Quick Reference

Program	Main purpose	Important output
Linear Regression	Predict a numeric value	Slope, intercept, MSE, R^2, graph
A* Search	Find a path using costs and heuristics	Path and total cost
SVM	Classify data into classes	Predicted class and accuracy
BFS / DFS	Traverse a graph	Traversal order

Before running a program, install the libraries it uses and copy each program into its own Python file. The regression program needs salary_data.csv in the same folder.